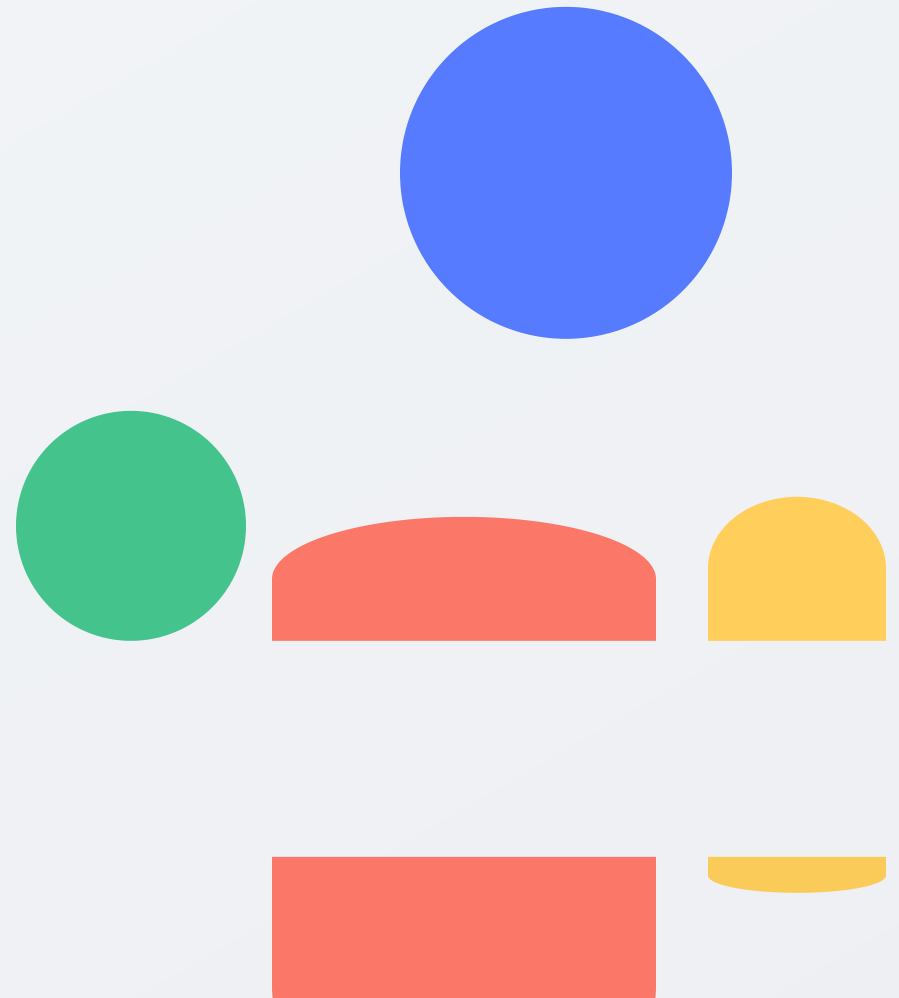


WEEK 6 · CH6 / CH25 · CONSTRUCTION & VERSION CONTROL

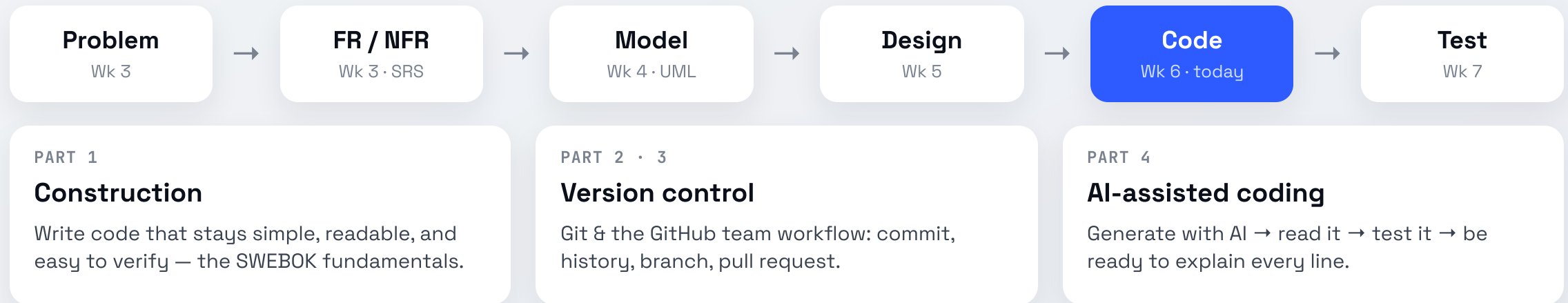
You designed it. Now *build* it — and track every change.

Last week you decided **how** the system is shaped. This week you turn design into **code** you can explain, and put every change under **version control** — Git & GitHub — so nothing is ever lost and every commit says *why*.



THE GOLDEN THREAD · ONE STEP FURTHER

Construction is where the design becomes real



Due this week: submit the **M3 Design Document** (part of the 12% M3 Design + Prototype) · **R2 — your response to the review (1%)**. The graded output is a **working prototype**, demoed live — there is no separate graded build.

SWEBOK V4 · CODE COMPLETE (MCCONNELL)

”Construction” is more than typing code

SWEBOK: construction is mostly **coding and debugging** — but also **detailed design, unit testing, and integration**. McConnell adds: it’s the **one activity guaranteed to happen on every project**.

Code is the real description

Diagrams drift; the **source code is often the only accurate, up-to-date description** of what the software really does.

Where quality is won or lost

Code is the ultimate deliverable — quality is built in *while you code*, not bolted on later.

- Design & write classes and routines
- Name variables & constants well
- Choose control structures; organize blocks
- Unit-test & debug your own code
- Review teammates’ code; have yours reviewed
- Format, comment & integrate the pieces

Links to **Configuration Mgmt** (construction makes the most items to track → Git), **Quality**, and **Management**. Reading: Sommerville Ch7, Ch25 · SWEBOK *Software Construction* · McConnell, *Code Complete*.

Five fundamentals of construction

These are the goals behind every “good practice” you’ll hear. The first three are what we work on today; the last two you already met (reuse a library; follow a style).

The one to remember: **minimize complexity** — a human has to read this code later, and that human is often you.

1 Minimize complexity

Simple, readable code > clever code. This drives almost every other rule.

2 Anticipate & embrace change

Build so the next change is easy — agile, small pieces, clear seams.

3 Construct for verification

Write code so bugs are *easy to find* — standards, logs, testable shape.

4 Reuse assets

Use existing libraries/components instead of reinventing — but only trusted ones.

5 Apply standards

Naming, layout, error handling — a shared style so the team reads as one.

FUNDAMENTAL #1

Simple & readable beats clever

✗ CLEVER – NOBODY CAN READ IT

```
def a(t,l): return [x for x in l
    if (x['d']-now()).days<=1 and not x['c']]
```

One line, cryptic names, does three things. A bug here is invisible.

✓ READABLE – ONE JOB, CLEAR NAMES

```
def tasks_due_soon(tasks):
    # due within 1 day and not done (FR-3)
    soon = []
    for task in tasks:
        days_left = (task.due_date - today()).days
        if days_left <= 1 and not task.done:
            soon.append(task)
    return soon
```

simple · 1 route



every if / loop adds a route



More routes through a function = more places a bug can hide, and more cases a test must cover.

Cyclomatic complexity (McCabe, 1976) is just a way to **count those routes** — more branches = harder to test and understand. So: fewer branches, smaller functions, clearer names. Rule of thumb: **one function does one thing**.

FUNDAMENTAL #5 · APPLYING STANDARDS (LIGHT)

Readable code & naming

DO

Name for meaning

`tasks_due_soon` , not `a` . A name should say what it is without a comment.

Small functions

If you can't see the whole function on one screen, split it.

One consistent style

Agree on naming/format across the team — let a formatter enforce it.

DON'T

Cryptic abbreviations

`t` , `l` , `tmp2` — future-you won't remember what they meant.

Comment the obvious, hide the tricky

Explain *why*, not *what*. Delete dead code, don't comment it out.

Copy-paste the same block 5×

Duplication = five places to fix the same bug. Make it a function.

This is *light* today — the point is habits, not a style rulebook. AI writes cleaner code when your prompt asks for readable names and small functions.

GOOD NAMES · BEFORE → AFTER

A name should tell you what it holds

❌ CRYPTIC	✅ CLEAR	WHY
a , x	tasks_due_soon	says what it holds
d	due_date	full word, no guessing
tmp2	filtered_tasks	describes the content
flag	is_overdue	a boolean reads yes/no
calc()	days_until_due()	verb + what it returns

Same logic, two names

```
# ❌ what does this even do?
def f(a, d):
    return [t for t in a if t.d < d]

# ✅ reads like a sentence
def tasks_due_before(tasks, cutoff):
    return [t for t in tasks
            if t.due_date < cutoff]
```

Rule of thumb: if you need a comment to explain a *name*, the name isn't done yet. Ask your AI for clear names in the prompt — it will follow.

Naming, done properly

1 Just-right length (Goldilocks)

Gorla & Benander (1990): debugging was easiest when names averaged **10–16 characters**. Not `x / tmp ; not`
`roll_no_of_a_student` .

2 Loop variables

`i , j , k` are fine for short loops. Used outside the loop or nested? Give it a real name — `record_count` .

3 Use opposites precisely

`begin/end , min/max , first/last , next/previous` — consistent pairs read faster.

Python naming conventions

KIND	STYLE	EXAMPLE
Variable / function	snake_case	<code>due_date</code> , <code>add_task()</code>
Class	PascalCase	<code>TaskRepository</code>
Constant	UPPER_SNAKE	<code>MAX_TASKS</code>
Module / package	lowercase	<code>tasks.py</code>

Code is **read far more than it's written** — favour read-time clarity. And tell your AI the convention in the prompt, so its draft matches your codebase.

FUNDAMENTAL #3 · A BRIDGE TO WEEK 7

Write code so bugs are easy to find

SWEBOK calls this **constructing for verification**: shape the code today so that you — and the tester next week — can find faults fast.

STRUCTURE

Testable shape

Small functions that take input and return output are easy to unit-test (Week 7).

STANDARDS

Follow conventions

Consistent style makes code reviews and reading fast — you spot the odd one out.

VISIBILITY

Log behaviour

Record what the program did, so when it breaks you can see *where*.

You're not testing yet — you're making next week's testing possible. Design → Code → **Test** is one continuous thread.

THE PROBLEM VERSION CONTROL SOLVES

You've already lived this pain

NAMING CHAOS

report_final_v2_FINAL_really.c

Which one is the real one? Version control gives you **one file** with a full history.

OVERWRITE

"You saved over my part!"

Two teammates edit the same file. Git **merges** both — or tells you exactly where they clash.

MYSTERY BUG

"It worked yesterday..."

Git shows **what changed, when, and by whom** — and lets you go back to a working version.

Version control is not optional in a real team. It's the **save button for the whole project's history**.

GIT = A TIMELINE OF SNAPSHOTS

Every commit is a snapshot with a reason

```
$ git log --oneline
a1b2c3d feat: add task with due date (FR-1)    ← Priya · 2h ago
9f8e7d6 fix: reject empty task title (FR-1)    ← Nan · yesterday
4c5b6a7 feat: mark task done (FR-2)           ← Aran · yesterday
0011223 chore: initial project structure      ← Priya · 2 days ago
```

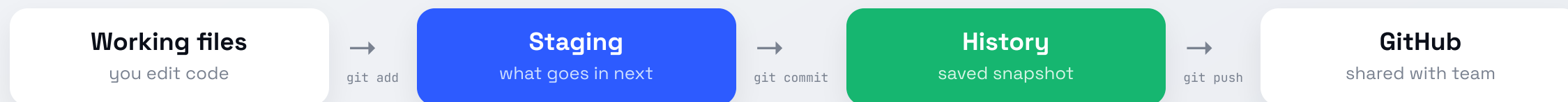
Each line answers **what changed**, **who**, **when**, and — from the message — **why**. This log *is* your project's memory.

IN PLAIN TERMS · A WORD YOU'LL MEET

SWEBOK calls each tracked file a **"configuration item"** — that just means *every file that makes up the product*: code, tests, docs. Keeping them all versioned together is a real engineering job, not just using a tool.

THE ONE LOOP YOU'LL REPEAT ALL SEMESTER

Edit → stage → commit → repeat



```
git status      # what did I change?
git add tasks.py # stage this file
git commit -m "feat: add task with due date (FR-1)"
git push        # send it to GitHub
```

IN PLAIN TERMS · THINK OF A BOX

`git add` = **put** the changes you want into a box · `git commit` = **seal & label** the box (one snapshot + a message that says why) · `git push` = **ship** the box to the team on GitHub. Keep boxes small and pack them often.

THE GOLDEN THREAD, IN YOUR COMMIT LOG

A good commit is small, and says *why*

✓ GOOD

```
feat: add task with due date (FR-1)

- validate the title is not empty
- store due_date as an ISO date
- return the new task id
```

One logical change. First line = type + what + **the FR it delivers**.

Rule for this course: **reference the FR** in the message — `feat: ... (FR-1)`. That's the Golden Thread reaching all the way into your history: Problem → FR → Design → **commit**.

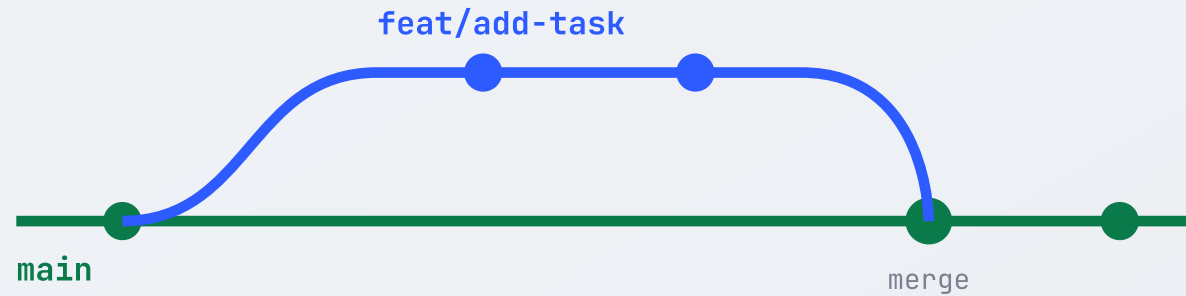
✗ BAD

```
stuff
fixed it
asdf
update (200 files changed)
```

Says nothing; bundles everything. A reviewer — and future-you — is lost.

WORK IN PARALLEL WITHOUT BREAKING MAIN

A branch is a safe parallel line



BRANCH

Split off

`git branch feat/add-task` — work without touching `main`.

COMMIT

Build there

Commit freely on your branch; `main` stays stable for everyone.

MERGE

Bring it back

When it's done and reviewed, merge into `main`.

IN PLAIN TERMS

A branch is like doing **"Save As"** on the whole project so you can experiment freely. If it works, you *merge* it back into `main`; if it doesn't, you throw the copy away and `main` was never touched.

WHEN TWO PEOPLE EDIT THE SAME LINES

A merge conflict is normal — read it, don't fear it

```
<<<<<< HEAD          # your version
    max_tasks = 50
=====
    max_tasks = 100      # your teammate's version
>>>>>> feat/limits    # the other branch
```

Git is telling you: “you both changed this line — **you** decide which is right.” Open the file, pick (or combine) the correct code, delete the `<<< / ≡ / >>>` markers, then commit the resolution.

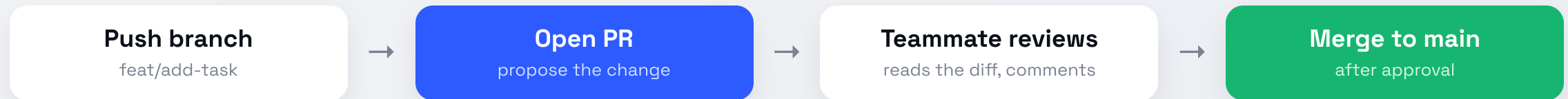
IN PLAIN TERMS

It's like two people editing the **same sentence** in one shared document. Git doesn't guess — it **pauses and asks you** which wording to keep. Nothing is lost or broken; you just choose, then save.

⚠ Never “fix” it with `git push --force` — that erases a teammate's work instead of resolving the clash. Never force-push a branch other people share.

THE TEAM'S QUALITY GATE

A Pull Request = "please review, then merge"



The PR shows the **diff** — exactly what lines changed. A teammate reads it before it reaches `main`. This is where **code review** lives (we go deeper in Week 9).

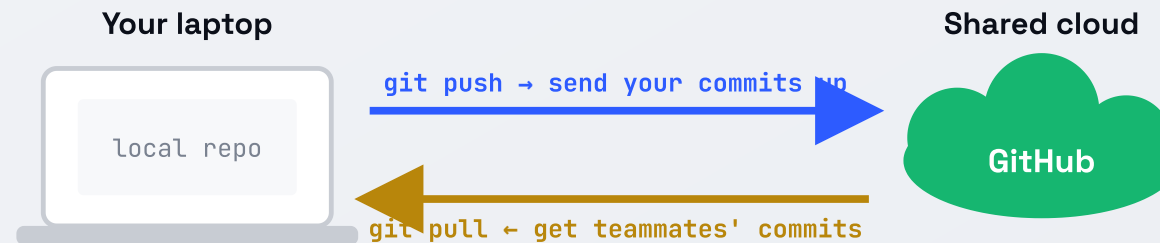
IN PLAIN TERMS

A Pull Request is like **handing your homework to a classmate to check** before you turn it in to the teacher (`main`). They read it, suggest fixes, and only then does it become part of the shared project.

Why it matters: no single person can quietly break `main`, and everyone learns the codebase by reading each other's changes.

GIT IS THE TOOL · GITHUB IS THE SHARED HOME

GitHub hosts the repo the whole team shares



`git clone` = your very first copy of the whole repo (you do it once) · after that you `push` up and `pull` down.

CLONE

Get a copy

Download the whole repo + its history to your machine.

PUSH

Send changes

Upload your commits so teammates can see them.

PULL

Get changes

Download teammates' latest commits into your copy.

PR

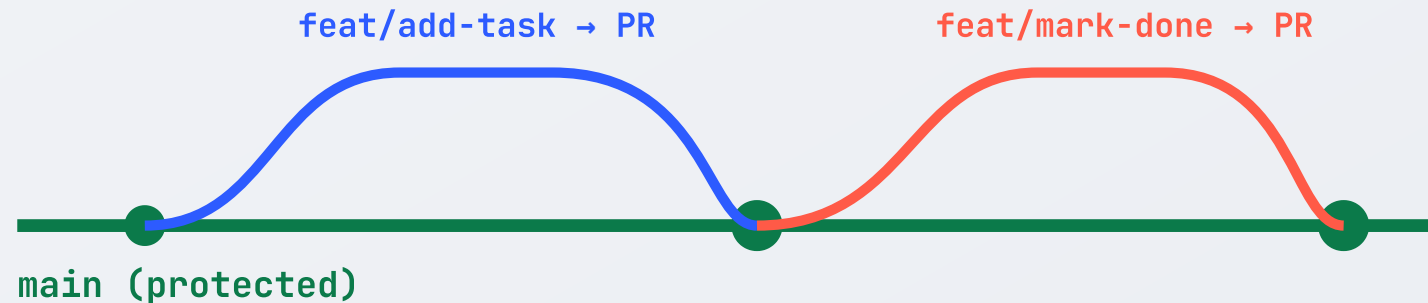
Review & merge

Propose, discuss, and approve changes in the browser.

Git works offline on your laptop. GitHub is where the team's single shared truth lives — plus PRs, issues, and history in the browser.

THE WORKFLOW YOUR TEAM USES FROM TODAY

One protected main, many short branches



RULE 1

Never commit to main directly

Every change arrives through a branch + PR.

RULE 2

Small branches, merge often

One feature per branch; don't let it drift for weeks.

RULE 3

Everyone commits

Each member's name appears in the history — that's your individual proof.

SWEBOK · SCM · WHAT TO TRACK, WHAT TO HIDE

main is your baseline — keep it clean

TRACK IN GIT

Source code & tests

The real deliverable and its safety net.

README, CLAUDE.md, docs

How to run it and how the AI should help.

Tag releases

Mark a working version — a *baseline* you can return to.

NEVER COMMIT

Secrets & API keys

Passwords, tokens, `.env` — once pushed, treat them as leaked.

Generated junk

`node_modules/`, build output — list them in `.gitignore`.

Personal data

Real user data belongs nowhere near a repo (PDPA — Week 9).

A `.gitignore` file tells Git what to leave out. Add it on day one, before the first push.

AI-FIRST · BUT YOU OWN EVERY LINE

Vibe coding, done like an engineer

AI writes the first draft. You are the engineer who reads it, tests it, and can explain it.



If you can't explain a line, it isn't done — it's a liability you'll own at the oral exam and in production.

FROM DESIGN TO A COMMITTED, VERIFIED CHANGE

The loop, end to end



If the test fails or you don't understand the code, **loop back to the prompt** — refine it, don't paste code you can't defend.

Your **AI Usage Log** records what you asked and what you changed — you bring it to the oral exam.

WHAT TO CATCH BEFORE YOU COMMIT

Five ways AI code goes wrong

FAILURE	WHAT IT LOOKS LIKE	HOW YOU CATCH IT
Hallucinated API	Calls a function or library that doesn't exist.	Run it — it errors. Check the real docs.
Missing edge cases	Works on the happy path; breaks on empty / null / huge input.	Test the boundaries yourself (Week 7).
Over-engineering	Adds config, layers, options no FR asked for.	Trace to a requirement — no FR, cut it.
Untrusted dependency	Suggests an obscure package to <code>pip install</code> .	SWEBOK: dependencies are supply-chain risk — verify before adding.
Silent wrong logic	Runs fine, gives the wrong answer.	Read it against the acceptance criteria.

AI is fast at the first draft and blind to your context. The reading, testing, and judgement are the parts **you** are graded on.

WORKED EXAMPLE · IMPLEMENT "ADD TASK" (FR-1)

Generate → read → commit

1 · The prompt (FR + rules)

```
Write add_task(title, due_date) for StudyMate.  
FR-1: a student adds a task with a  
title and a due date.  
- title must not be empty → ValueError  
- due_date is ISO 'YYYY-MM-DD'  
- return the new task as a dict + id  
Keep it simple and readable.
```

2 · Read the draft, then commit

```
def add_task(title, due_date):  
    if not title.strip():  
        raise ValueError("title required")  
    return {  
        "id": next_id(),  
        "title": title,  
        "due_date": due_date,  
        "done": False,  
    }
```

3 · `git commit -m "feat: add task with due date (FR-1)"`

Notice: you can explain *every* line — the empty-title check maps straight to FR-1's acceptance criterion. That's the standard.

KEEP THIS OPEN DURING THE STUDIO

The commands you need today

```
# one time, per project
git init
git remote add origin <github-url>
# create .gitignore first!

# the everyday loop
git status
git add <file>
git commit -m "feat: ... (FR-1)"
git push
```

```
# branching
git branch feat/add-task
git checkout feat/add-task
# ...commit your work...
git push origin feat/add-task
# then open a Pull Request on GitHub

# get teammates' latest
git pull
git log --oneline    # see history
```

First time on a machine: set your name once — `git config --global user.name "Your Name"` and `user.email`. Then your commits are attributed to you.

TEAM STUDIO · ~55 MIN · THIS STARTS M4 BUILD

Set up the repo & land the first real commit

STATION A · 15 MIN

Create the repo

One member makes the GitHub repo, adds a `.gitignore` + `README`, invites the team.

Every member clones it.

STATION B · 25 MIN

Implement one FR with AI

Pick one FR. Prompt AI → read the draft → run it → fix names/edge cases. Explain it to each other.

STATION C · 15 MIN

Everyone commits

Each member makes ≥ 1 commit with a message like `feat: ... (FR-#)`, then `push`. Check the history shows all names.

Leave with: a live GitHub repo, one FR implemented, and a commit from **every** member. This **hardens your M3 prototype** — construction & Git are skills you apply, assessed on the written exam, not a separately graded build.

DELIVERABLES · WEEK 6

What's due, and how R2 works

SUBMIT · W6

M3 Design Document

Architecture + class/sequence diagrams + one screen wireframe + one ADR. Part of **M3 Design + Prototype (12%)**.



DUE

R2 — response (1%)

Reply to the R1 review you received: per issue, **changed** (say what) or **rejected with a reason**.



BUILD TOWARD · W10

Working Prototype

The course **output** — a prototype a user can try, demoed live for the **Prototype Demo (3%)**.

Construction/Git are understand-only skills to get there.



R2 is not "we fixed everything" — a reasoned rejection is a valid, graded engineering response. Address every issue the reviewers raised.

🧠 AI CRITIQUE #6 · LAST 10 MIN · ON A4

Spot it · explain it · fix it

“Just put the API key straight in the code and commit it — `API_KEY = "sk-live-9f2a..."` . The repo is private, so it's fine.”

— an AI coding assistant, confidently

1 · SPOT

What's wrong?

A real secret (`sk-live-...`) is going into a commit — and into the shared history. Is “private repo” actually safe?

2 · EXPLAIN

Why is it wrong?

Once pushed, treat the key as leaked. Git keeps it in history even after you delete the line — and teammates, forks & a future public repo all carry it.

3 · FIX

The right way

Put it in `.env` , add `.env` to `.gitignore` , load from the environment. Already pushed? **Revoke/rotate the key now** — deleting the line is not enough.

Hand in your A4 before you leave — this is your attendance + AI-critique mark for the week.

WEEK 6 • IN FOUR LINES

Build it small, track it all, own every line

CONSTRUCTION

Simple, readable, verifiable

Minimize complexity, name for meaning, and shape code so bugs are easy to find.

GITHUB

One protected main, many short branches

Everyone commits; nothing reaches main without a review.

Due this week: M3 Design Document • R2 response (1%). The graded output is a working prototype (demoed live); construction, Git & testing are understand-only skills on the exams — no graded repo. Next: Week 7 — Unit testing (concept).

VERSION CONTROL

Every change is a commit that says why

Small commits, FR reference in the message, branch + PR — Git is the project's memory.

AI-FIRST

Generate → read → test → explain

AI drafts; you're the engineer. Never commit code you can't defend.